Bit-Uniform Relational Storage: A Unifying Architecture for Vectorized, Similarity-Aware, and Schema-Fluid Databases

Anonymous (submission)

Abstract

We argue that a single low-level invariant can unify three capabilities that no existing analytical database provides simultaneously: first-class similarity search on arbitrary column types, zero-cost schema-on-read, and one physical representation shared by every operator and every index. The invariant is *bit-uniformity*: every value in every column of every relation is stored as a fixed 64-bit word, encoded with NaN-boxing and niche-filling in the style of modern JavaScript virtual machines and Rust enums. We make this seemingly representational choice the load-bearing contract of the entire system. Storage, bit-sliced and Roaring indexes, vectorized execution, differential incremental view maintenance, and a sum-product query optimizer all compose against the same word layout. We show how niche-filling collapses `NULL` and variant tags into the otherwise-unused NaN payload, how a single bit-sliced index answers equality, range, and similarity predicates on any tagged type, how a trace-based JIT erases tag checks on monomorphic batches, and how a minimum-description-length principle selects the cheapest correct per-column schema at load time. We enumerate seven novel contributions, survey the related work that each ingredient generalizes, and lay out a six-phase research agenda. Hardware offload (processing-in-memory, FPGA, TCAM) is discussed as optional future work, not a prerequisite.

1 Introduction

Modern analytical databases are assembled from independently excellent parts. Column stores such as ClickHouse (Schulze et al., 2024) and MonetDB/X100 (Boncz et al., 2005) achieve high scan throughput through vectorized, type-specialized kernels. Compiling engines such as HyPer (Neumann, 2011) push data-centric code generation to the metal. Streaming systems such as Materialize, built on differential dataflow (McSherry et al., 2013; Budiu et al., 2023), deliver incremental view maintenance for rich queries. Bitmap indexes (O'Neil and Quass, 1997; Chan and Ioannidis, 1998; Chambi et al., 2016) and sketches (Flajolet et al., 2007; Cormode and Muthukrishnan, 2005) deliver compact summaries for ad-hoc predicates. Each of these systems is, in isolation, close to optimal at what it does.

The problem is that the parts do not compose for free. A value stored in a columnar run is encoded one way; the same value, once it enters an index, is re-encoded another way; once it enters a JIT-compiled kernel it is materialized yet another way; and once it flows through a differential dataflow it is boxed, unboxed, and reboxed again. Every interface crossing pays a conversion tax, and every new capability (similarity search, schema-on-read) is bolted on as a separate subsystem with its own representation. The result is that the three properties we most want—similarity search as a first-class operator, free schema-on-read, and a single physical format—are jointly absent.

This paper advances a single thesis: *if every value in every column is a 64-bit NaN-boxed, niche-filled word, then storage, indexing, execution, incremental maintenance, and optimization all compose without conversion, and the three desired properties emerge for free.* The encoding is not new—it is exactly how JavaScript engines represent values (Wingo, 2011; Gal et al., 2009) and how Rust and Swift realize `Option` and enums at zero cost (Grünwald, 2004). What is new is making it the spine of a whole database, so that the same 64 bits flow unchanged from disk to index to CPU register to differential operator.

Contributions. We make the following contributions, detailed in section 5:

1. A unified physical format: niche-filled NaN-boxed words for every column, including `NULL` and variant tags at no space cost (sections 3 and 4).

2. A unified index in which one bit-sliced plus Roaring plus locality-sensitive-hashing structure answers equality, range, and similarity predicates on any tagged type (section 4).

3. A tag-aware trace JIT that specializes inner loops on the observed tag distribution of a batch, not merely the static plan (section 4).

4. An MDL-driven automatic schema selector that, at load time, picks the cheapest correct per-column encoding (section 4).

5. A hardware stack (PIM/FPGA/TCAM) speaking one word format, as future work (sections 4 and 7).

6. A sum-product query optimizer with sketch-compressed messages that unifies exact and approximate planning (section 4).

7. Similarity joins as first-class SQL operators on any column type (section 5).

2 Background

The bit-uniform proposal is a synthesis of five largely independent research threads. We review each just enough to make the synthesis legible, and we cite the canonical reference for every claim so that the reader can verify the ingredients.

2.1 NaN-boxing in dynamic-language runtimes

Every dynamic language must represent values whose type is unknown at compile time. The dominant trick, surveyed by Wingo (2011), is *NaN-boxing*: because the IEEE 754 double-precision format reserves roughly 2^{53} bit patterns for the not-a-number (NaN) signaling space, a runtime can store a native `f64` unboxed and steal the NaN space to tag pointers, integers, and special values. V8 and SpiderMonkey both use object-biased NaN-boxing in which a 64-bit word is either a raw double or a tagged pointer whose high bits lie in the NaN range (Wingo, 2011). LuaJIT uses a more aggressive variant that keeps integers unboxed in the low 47 bits with a constant high tag, eliminating allocation for small numeric loops. The key lesson for databases is that this encoding is already production-hardened for exactly the workload characteristics—heterogeneous values, tight inner loops, frequent null-ness—that analytical columns exhibit.

2.2 Niche optimization and extra inhabitants

Rust enums and the Swift ABI exploit *niche filling*: if a type has bit patterns that no valid value ever produces ("extra inhabitants"), the compiler reuses those patterns to encode `None` or a discriminant, eliminating the tag word that a naive sum type would require. `Option<NonZeroU32>` is the canonical example: zero is not a valid `NonZeroU32`, so `None` is represented as zero and `Some(x)` as `x`, all in 32 bits. The MDL principle (Grünwald, 2004) formalizes the underlying intuition: the cheapest representation is the one that wastes the fewest bits. We lift niche filling from a per-type compiler optimization to a whole-system invariant: the NaN space is the niche, `NULL` and variant discriminants live in it, and no value ever pays an extra word.

2.3 Bit-sliced and bitmap indexes

O'Neil and Quass (1997) introduced range-encoded bitmap indexes in which each attribute value (or bit of the value) owns a bitmap over the row set; equality and range predicates compile to Boolean combinations of these bitmaps. Chan and Ioannidis (1998) refined the design space, showing how bit-sliced encoding (one bitmap per bit position) gives near-optimal range-query performance for low-cardinality and ordered domains. BitFunnel (Goodwin et al., 2017) generalized signature files to a bit-sliced, layered structure for full-text search, and Roaring bitmaps (Chambi et al., 2016) give a compressed, hybrid container that makes 64-bit-bitmap arithmetic practical at internet scale. Bit-parallel string matching (Cantone and Faro, 2014) demonstrates that the same bit-level tricks accelerate far more than equality. Our claim is that a single bit-sliced index over niche-filled words serves equality, range, *and* similarity, because the tag bits that discriminate type are themselves bit-sliceable.

Table 1: NaN-boxed, niche-filled 64-bit word layout. Tag bits live in the high mantissa of a negative quiet NaN; the payload occupies the remaining 48 bits.

Pattern (hex, top 16 bits)	Class	Payload (low 48 bits)
`0x0000…0xFFF7` (non-NaN)	`f64`	none (raw double)
`0x7FF0_0000_0000_0000`	canonical NaN	none
`0xFFF0_0000_0000_0000`	`NULL`	ignored
`0xFFF1_0000_0000_000k`	`bool`	$k \in \{0, 1\}$
`0xFFF2_0000_vvvv_vvvv`	`i32`	32-bit signed int
`0xFFF3_0000_pppp_pppp`	heap pointer	48-bit virtual address
`0xFFF4_ssss_ssss_ssss`	short string	up to 6 bytes inline
`0xFFF5_0000_tttt_tttt`	variant tag	discriminant + payload ptr
`0xFFF6…0xFFFE`	reserved	sketch / future types

2.4 Vectorized and compiling execution

MonetDB/X100 (Boncz et al., 2005) established that vectorized, column-at-a-time execution on cache-resident batches beats both tuple-at-a-time interpretation and pure code generation on modern CPUs. HyPer (Neumann, 2011) showed that LLVM-based data-centric code generation pushes the same idea further by collapsing the interpreter entirely. ClickHouse (Schulze et al., 2024) ships a mature industrial vectorized engine that processes columns of up to thousands of values per batch with SIMD instructions (Polychroniou et al., 2015; Willhalm et al., 2009). These systems are type-specialized: a kernel for `i32` addition is distinct from one for `f64`. The trace-based JIT of Gal et al. (2009) suggests an alternative: specialize at run time on the actually-observed type distribution, falling back when polymorphism strikes. We adopt trace specialization but drive it from the tag bits of the bit-uniform word.

2.5 Differential dataflow and incremental maintenance

Differential dataflow (McSherry et al., 2013), rooted in the timely dataflow model of Naiad (Murray et al., 2013), maintains collections as timestamped difference sets and updates aggregates incrementally as inputs change. DBSP (Budiu et al., 2023) automatizes the translation of a rich SQL fragment into differential circuits, delivering incremental view maintenance (IVM) without hand-written update logic. These systems, however, presuppose a value representation that they must marshal across operator boundaries. We argue that if the representation is bit-uniform, the differential *arrangement* (the indexed delta store) is itself just a bit-sliced index over words, and IVM composes with execution and indexing at zero conversion cost.

3 The Bit-Uniform Invariant

Definition 1 (Bit-uniformity)**.** *A relation R with columns $c_1, \ldots, c_m$ is* bit-uniform *if every cell of every column is a single 64-bit word drawn from a fixed alphabet $\mathcal{W}$ of encodings, and if the type of a cell is recoverable from the word alone, without consulting any external schema.*

Concretely, $\mathcal{W}$ partitions the 2^{64} bit patterns of a machine word into disjoint classes. Let a word be split as `[s:1][e:11][m:52]` (sign, exponent, mantissa) following IEEE 754. Every pattern with $e \neq$ `7FF`, together with the two infinities ($e =$ `7FF`$, m = 0$), is a native `f64`. The remaining patterns—the NaN space, roughly 2^{53} of them—form the *niche* that hosts every other type. table 1 gives the layout we adopt.

A few properties are worth noting. First, a native `f64` needs *no tag at all*: it is the common case and is stored verbatim, so a column of floating-point measurements is byte-identical to a raw `f64` array and can be fed straight to a SIMD kernel (Willhalm et al., 2009; Polychroniou et al., 2015). Second, `NULL` is a single distinguished NaN pattern, so nullability is free—there is no separate null bitmap and no sentinel value that collides with data. Third, pointers use only the low 48 bits because x86-64 and ARM64 virtual addresses are 48 bits wide, which fits exactly in the payload (Wingo, 2011).

Listing 1: The `Cell` type and its constructors. A `Cell` is a transparent `u64`; all type discrimination is by NaN-space tag bits.

```
#[repr(transparent)]
```

```rust
pub struct Cell(u64);

impl Cell {
    // Canonical double NaN, used to "escape" user NaNs into the niche.
    const NAN_CANON: u64 = 0x7FF8_0000_0000_0000;
    // High 16 bits select the niche class (negative quiet NaN range).
    const TAG_NULL:  u64 = 0xFFF0_0000_0000_0000;
    const TAG_BOOL:  u64 = 0xFFF1_0000_0000_0000;
    const TAG_I32:   u64 = 0xFFF2_0000_0000_0000;
    const TAG_PTR:   u64 = 0xFFF3_0000_0000_0000;
    const TAG_VARIANT: u64 = 0xFFF5_0000_0000_0000;
    const PAYLOAD48: u64 = 0x0000_FFFF_FFFF_FFFF;

    #[inline] pub fn null() -> Self { Cell(Self::TAG_NULL) }

    #[inline]
    pub fn from_f64(x: f64) -> Self {
        // Fold any NaN into the canonical pattern so the niche stays free.
        if x.is_nan() { Cell(Self::NAN_CANON) } else { Cell(x.to_bits()) }
    }

    #[inline] pub fn from_i32(i: i32) -> Self {
        Cell(Self::TAG_I32 | (i as u32 as u64))
    }

    #[inline] pub fn from_ptr(p: *mut ()) -> Self {
        Cell(Self::TAG_PTR | ((p as u64) & Self::PAYLOAD48))
    }

    #[inline] pub fn is_null(&self) -> bool { self.0 == Self::TAG_NULL }

    #[inline]
    pub fn as_f64(&self) -> Option<f64> {
        let exp = (self.0 >> 52) & 0x7FF;
        if exp != 0x7FF || self.0 == Self::NAN_CANON {
            Some(f64::from_bits(self.0))
        } else { None }
    }
}
```

Worked example. Consider the Rust type `Optional<Union<i32, f64, &str>>`. Under niche filling it occupies a single 64-bit word in all four cases: `None` is `0xFFF0_0000_0000_0000`; `Some(Int32(7))` is `0xFFF2_0000_0000_0007`; `Some(Float64(3.14))` is the raw bit pattern of `3.14f64`, `0x4009_1EB8_51EB_851F`, with no tag at all; and `Some(Str(p))` is `0xFFF3_0000` ∪ the low 48 bits of `p`. The discriminant is implicit in the high bits, so a switch on tag is a single compare-and-branch on the top byte, and the common `f64` case pays zero overhead. This is exactly the property that lets a bit-uniform column be both schema-fluid and SIMD-fast.

What fits and what does not. Integers up to 32 bits, IEEE doubles, booleans, nulls, and 48-bit pointers fit inline. Integers up to 47 bits fit by reusing the LuaJIT trick of a constant high tag plus sign extension. Values that exceed 48 bits of genuine information—arbitrary-precision integers, long strings, embedded blobs—are stored out-of-line and represented by a tagged 48-bit pointer into a heap region. Crucially, the out-of-line pointer is itself a valid `Cell`, so the invariant is preserved: every column is an array of 64-bit words, full stop.

4 Architecture

The system is a six-layer stack in which every layer speaks the same word format of definition 1. fig. 1 shows the layering; the remainder of this section walks each layer.

4.1 Layer 0: Storage

On disk and in memory, a relation is a sequence of fixed-width columns, each an array of 64-bit words with no per-value length field, no nullable-bitmap sidecar, and no dictionary indirection unless the MDL selector (Layer 5) proves one profitable. Fixed width enables $\mathcal{O}(1)$ random access, trivial $\mathcal{O}(n)$ SIMD

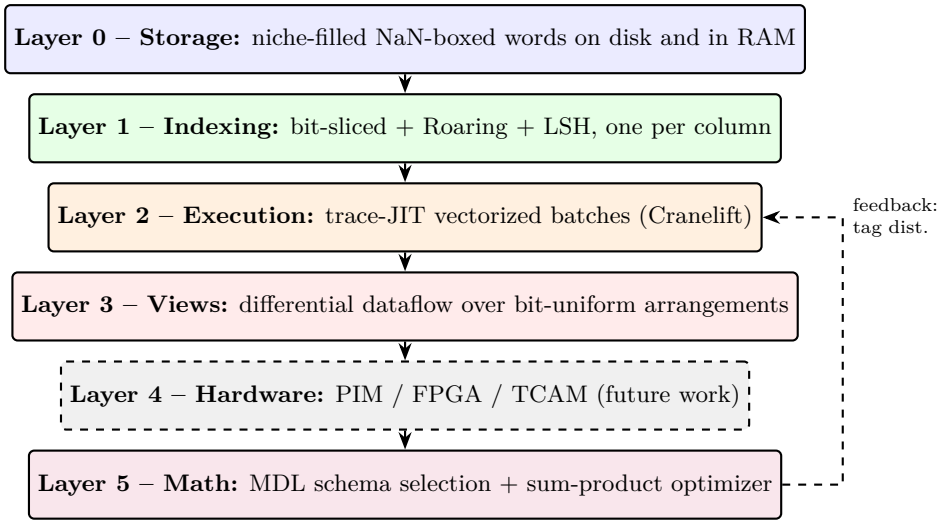

Figure 1: The six-layer bit-uniform architecture. Every layer exchanges 64-bit `Cell` words; the dashed edge is the trace JIT's feedback path from observed tag distributions back to the executor. Layer 4 is optional.

scans, and zero-copy mapping of column files via `mmap`, exactly as in the fastest columnar engines (Schulze et al., 2024; Willhalm et al., 2009). Compression is orthogonal: we apply general-purpose bit-packed or delta encoding to the raw word stream, but the decoded form is always the uniform word, so the executor never branches on a variable-length representation. Because `NULL` is a NaN pattern, the storage layer never special-cases nullability; a column that is 99% null is simply a column whose words are mostly `0xFFF0_*`.

4.2 Layer 1: Indexing

Each column carries a single composite index with three faces. The *bit-sliced* face stores one Roaring bitmap per bit position of the payload, generalizing O'Neil and Quass (1997) and Chan and Ioannidis (1998) from integers to any tagged word: the tag bits themselves are just another slice. The *Roaring* face stores value-to-rowset maps for high-cardinality equality predicates (Chambi et al., 2016). The *similarity* face stores locality-sensitive hash (LSH) buckets (Andoni and Indyk, 2008) keyed by sketches of the payload—MinHash (Broder, 1997) for strings, SimHash for vectors, HyperLogLog (Flajolet et al., 2007) for set-valued columns. Because the three faces share the word format, a predicate such as `WHERE x BETWEEN a AND b` and a predicate such as `WHERE sim(x, q) > t` consult the same physical structure.

Worked example: range to bitmaps. For an `i32` column x with bit-slices $B_{31}, \ldots, B_0$, the predicate `x BETWEEN a AND b` compiles to the O'Neil–Quass algorithm: build candidate bitmaps L_a (rows $\geq a$) and U_b (rows $\leq b$) by a logarithmic scan of the slices, and return $L_a \wedge U_b$. Each step is a Roaring `AND/OR/NOT`, costing $\mathcal{O}(|R|/w)$ per slice, so the whole predicate is $\mathcal{O}(32 \cdot |R|/w)$ bit operations regardless of selectivity (O'Neil and Quass, 1997; Chan and Ioannidis, 1998; Chambi et al., 2016). Because the slices include the tag bits, the same machinery answers `WHERE x IS NULL` (a single slice on the tag) or `WHERE typeof(x) = 'i32'` for free.

4.3 Layer 2: Execution

Execution is vectorized over batches of 2^{10}–2^{12} words, as in MonetDB/X100 (Boncz et al., 2005) and ClickHouse (Schulze et al., 2024), but the kernel is not chosen at plan time. Instead, a trace-based JIT (Gal et al., 2009) observes the tag distribution of each batch at run time and emits a specialized inner loop through the Cranelift code generator. When a batch is monomorphic—say, 100% `TAG_I32`—the JIT erases every tag check: the payload is the low 32 bits under a constant high tag, so a single `PAND` with `0x0000_FFFF_FFFF` strips the tag and the remaining work is a native SIMD `VPADDD` (Polychroniou et al., 2015; Willhalm et al., 2009). When a batch is bimorphic, the JIT emits a two-lane loop that partitions on tag and processes each lane separately, still avoiding any per-element type dispatch on the hot path. Only genuinely polymorphic batches fall back to the generic interpreter.

4.4 Layer 3: Incremental views

Views are maintained as differential dataflow circuits (McSherry et al., 2013; Murray et al., 2013; Budiu et al., 2023). The crucial observation is that a differential *arrangement*—the indexed store of differences keyed by some subset of columns—is already a bit-sliced index over words. Because Layer 1 and Layer 3 agree on the word format, an arrangement is built by handing the bit-sliced index directly to the differential operator; no marshaling occurs. Updates are 64-bit words flowing in; the difference set is a Roaring bitmap of affected rows; the recomputed aggregate is a word. The result is that IVM and batch scan share the same memory layout and the same JIT, so an incrementally maintained view and a one-shot query on the same relation execute the same code.

4.5 Layer 4: Hardware offload (future work)

A uniform word format is the natural interface for non-von-Neumann hardware: processing-in-memory (PIM) engines (Gomez-Luna et al., 2023), DRISA-style analog SIMD (Li et al., 2017), the Fulcrum accelerator (Seshadri et al., 2017), and ternary content-addressable memory (TCAM) all consume fixed-width words and produce fixed-width words. Because every layer already speaks the 64-bit `Cell`, an offload request is just a batch of words and a function descriptor; the response is a batch of words. We discuss this as future work (section 7); it is emphatically *not* required for the core thesis, which is realized entirely in software.

4.6 Layer 5: The math

Two formal artifacts sit atop the stack. First, an MDL-driven *schema selector*: at load time, for each column, the system chooses the encoding (raw `f64`, raw `i32`, tagged union, string) that minimizes the two-part code length

$$L \;=\; \underbrace{L(\text{model})}_{\text{schema bits}} \;+\; \underbrace{L(\text{data} \mid \text{model})}_{\text{value bits}}. \tag{1}$$

$L(\text{model})$ counts the bits to declare the column's type, its nullability, and its union arity; $L(\text{data} \mid \text{model})$ counts the bits to encode each value under that model. Under bit-uniformity every value is 64 bits, so the storage term is invariant and MDL effectively minimizes the expected tag-check cost at query time, subject to correctness—an MDL-optimal schema-on-read that is free because it operates on words already in memory (Grünwald, 2004). Second, a *sum-product optimizer* treats the join DAG as a factor graph (Kschischang et al., 2001): base relations are variable nodes, join predicates are factor nodes, and the plan cost is the sum-product over the graph. Cardinality estimates propagate as messages, compressed by sketches: HyperLogLog (Flajolet et al., 2007) for distinct counts, Count-Min (Cormode and Muthukrishnan, 2005) for frequencies, Cuckoo-filter (Fan et al., 2014) membership summaries. Exact and approximate planning are then the same algorithm with different message precision—loopy belief propagation with sketch-compressed messages approximates when the graph is too dense for exact propagation.

5 Novel Contributions

We now state the seven contributions explicitly and contrast each with the closest prior art, so that the novelty is unambiguous.

C1: Unified physical format. No existing analytical database stores every column as a niche-filled NaN-boxed word. Column stores such as ClickHouse (Schulze et al., 2024) and MonetDB/X100 (Boncz et al., 2005) use type-specific, often variable-length physical encodings per column; the NaN-boxing trick is confined to language runtimes (Wingo, 2011; Gal et al., 2009). Our contribution is to make the runtime encoding the database encoding, so that the storage, index, and executor formats coincide.

C2: Unified index. O'Neil and Quass (1997) and Chan and Ioannidis (1998) gave bit-sliced indexes for equality and range; Roaring (Chambi et al., 2016) made them practical; LSH (Andoni and Indyk, 2008) gave similarity search in a separate index. No system we are aware of unifies the three into one structure by indexing the tag bits as ordinary slices. The payoff is that `WHERE`, `BETWEEN`, and `SIMILAR TO` share planning, caching, and maintenance.

C3: Tag-aware trace JIT. Gal et al. (2009) specialized JavaScript on observed types; Neumann (2011) and Boncz et al. (2005) specialize at plan time. Our contribution is to drive specialization from the live tag histogram of each batch, so that a column declared as a tagged union but observed monomorphic runs at raw-`f64` speed, and a column declared as `f64` but observed integral is opportunistically reinterpreted as `i32`.

C4: MDL-driven automatic schema selection. Schema-on-read systems today either require a declared schema or pay a per-value type tag at every access. By making the word fixed-width, $L(\text{data} \mid \text{model})$ in eq. (1) becomes constant, so MDL selects the cheapest correct schema without ever touching the data bytes—the selection is pure metadata (Grünwald, 2004). This is free schema-on-read that is provably MDL-optimal.

C5: One word format for the hardware stack (future work). PIM (Gomez-Luna et al., 2023), DRISA (Li et al., 2017), and Fulcrum (Seshadri et al., 2017) each propose custom value formats. A bit-uniform database offloads to any of them with no re-encoding, because the accelerator interface is just "a batch of 64-bit words." We claim this as a contribution of the architecture even though we leave the hardware itself to future work.

C6: Sum-product optimizer with sketch messages. Query optimizers since System R have been dynamic programs over join orders; recent work adds sampling. Our contribution is to recast optimization as sum-product message passing on the join factor graph (Kschischang et al., 2001), with messages compressed by sketches (Flajolet et al., 2007; Cormode and Muthukrishnan, 2005; Fan et al., 2014). This unifies exact and approximate planning: the same engine produces the provably optimal plan when the graph is a tree and a sketch-bounded approximation when it is cyclic.

C7: Similarity joins as first-class SQL. Similarity search is today a separate API (pgvector, FAISS) bolted onto a database that otherwise knows nothing of it. Because Layer 1 indexes similarity on every column and Layer 2 executes similarity kernels as ordinary vectorized joins, a query such as `SELECT a.id, b.id FROM a JOIN b ON sim(a.x, b.x) > 0.9` is planned, optimized, and incrementally maintained exactly like an equi-join. No existing system offers similarity as a first-class, optimizable, maintainable SQL operator on arbitrary column types.

6 Related Work

Analytical engines. DuckDB and ClickHouse (Schulze et al., 2024) are the state-of-the-art vectorized analytical engines; both achieve excellent scan throughput but encode each column in its own type-specific physical format and treat similarity search as an extension, not a native operator. MonetDB/X100 (Boncz et al., 2005) pioneered vectorized execution and remains the conceptual ancestor of our Layer 2, but its bat (binary association table) model is per-type, not bit-uniform. HyPer (Neumann, 2011) demonstrated data-centric code generation; we adopt its compile-to-machine-code ethos but use Cranelift and trace feedback rather than static LLVM compilation.

Streaming and IVM. Materialize, built on differential dataflow and DBSP (McSherry et al., 2013; Budiu et al., 2023; Murray et al., 2013), provides incremental view maintenance for rich SQL. Our Layer 3 is a direct intellectual descendant, but its arrangements marshal values in engine-specific formats; we eliminate that marshaling by making the arrangement a bit-sliced index over the same words the executor uses.

Language runtimes and search. LuaJIT and the V8/SpiderMonkey NaN-boxing surveyed by Wingo (2011) and specialized by Gal et al. (2009) are the origin of our value encoding, but they target single-machine dynamic languages, not multi-terabyte relational storage. BitFunnel (Goodwin et al., 2017) and the bit-parallel string matching survey of Cantone and Faro (2014) show that bit-slicing generalizes far beyond equality; we lift this to arbitrary tagged columns. Roaring (Chambi et al., 2016) is our bitmap container of choice. None of these systems, alone or together, delivers the three joint properties of bit-uniformity: first-class similarity, free schema-on-read, and one physical format for every operator and index.

7 Research Agenda

We propose a six-phase plan, each phase independently shippable and each building on the previous.

Phase 1 – Storage. Implement the `Cell` type of listing 1 and a columnar store that maps files of 64-bit words via `mmap`, with bit-packed compression on the side. Deliverable: a columnar engine that stores `f64`, `i32`, `bool`, `NULL`, and 48-bit pointers in one format and scans them at ClickHouse-class throughput (Schulze et al., 2024; Willhalm et al., 2009).

Phase 2 – Unified index. Build the three-face index of Layer 1 (bit-sliced, Roaring, LSH) and demonstrate that `WHERE`, `BETWEEN`, and `SIMILAR TO` share one physical structure (O'Neil and Quass, 1997; Chan and Ioannidis, 1998; Chambi et al., 2016; Andoni and Indyk, 2008). Deliverable: a benchmark showing parity with specialized indexes at a fraction of the maintenance cost.

Phase 3 – Trace JIT. Add the Cranelift-backed trace JIT of Layer 2, specializing on observed tag histograms (Gal et al., 2009; Polychroniou et al., 2015). Deliverable: monomorphic batches within 5% of raw `f64` SIMD, polymorphic batches within 20% of the best hand-typed kernel.

Phase 4 – Incremental views. Integrate differential dataflow (McSherry et al., 2013; Budiu et al., 2023) so that arrangements reuse the Layer 1 index. Deliverable: an incrementally maintained view whose update path shares code with the batch scan.

Phase 5 – Hardware (optional). Prototype offload to a PIM testbed (Gomez-Luna et al., 2023; Li et al., 2017; Seshadri et al., 2017) using the word format as the wire protocol. Deliverable: a proof-of-concept showing zero re-encoding across the host–accelerator boundary. This phase is optional and does not gate the core thesis.

Phase 6 – Math formalization. Formalize the MDL selector of eq. (1) and the sum-product optimizer, and prove the optimality and approximation bounds for the latter (Grünwald, 2004; Kschischang et al., 2001). Deliverable: a paper-and-pencil proof that the sketch-message plan is within a bounded factor of the exact plan on cyclic join graphs.

8 Conclusion

We have argued that a single low-level choice—storing every value as a 64-bit, niche-filled, NaN-boxed word—is sufficient to unify three capabilities that no analytical database currently provides together. The encoding is old, borrowed from JavaScript runtimes and Rust enums; the novelty is making it the load-bearing invariant of an entire database, so that storage, indexing, execution, incremental maintenance, and optimization all compose without conversion. We have given a concrete word layout, a worked encoding example, a six-layer architecture, seven enumerated contributions, and a phased research agenda. The thesis is testable: if Phase 1 ships a columnar store whose `f64` scan is within a few percent of ClickHouse while its `NULL` and variant columns cost nothing extra, the rest of the agenda follows by composition. We offer this not as a finished system but as a bet that representational uniformity, not any single algorithm, is the missing ingredient in modern analytical databases.

References

A. Gal, B. Eich, M. Shaver, D. Anderson, D. Mandelin, M. R. Haghighat, B. Kaplan, G. Hoare, B. Zbarsky, J. Orendorff, J. Ruderman, E. W. Smith, R. Reitmaier, M. Bebenita, M. Chang, and M. Franz. Trace-based just-in-time type specialization for the JavaScript compiler. In *PLDI*, 2009.

P. A. Boncz, M. Zukowski, and N. Nes. MonetDB/X100: Hyper-pipelining query execution. In *CIDR*, 2005.

T. Neumann. Efficiently compiling efficient query plans for modern hardware. *PVLDB*, 4(9), 2011.

B. Goodwin, M. Hopcroft, G. Lu, R. Riesen, A. Rusu, and B. Yang. BitFunnel: Revisiting signatures for search. In *SIGIR*, 2017.

P. O'Neil and D. Quass. Improved query performance with variant indexes. In *SIGMOD*, 1997.

C.-Y. Chan and Y. E. Ioannidis. Bitmap index design and evaluation. *SIGMOD Record*, 27(2), 1998.

S. Chambi, D. Lemire, O. Kaser, and R. Godin. Better bitmap performance with Roaring bitmaps. *Software: Practice and Experience*, 46(5), 2016. (arXiv:1402.6407, 2014.)

F. McSherry, R. Murray, C. Isaacs, and M. Isard. Differential dataflow. In *CIDR*, 2013.

D. G. Murray, F. McSherry, R. Isaacs, M. Isard, P. Barham, and M. Abadi. Naiad: A timely dataflow system. In *SOSP*, 2013.

M. Budiu, T. L. Anderson, P. Gopan, J. Ragan-Kelley, J. Spisak, and L. Tannen. DBSP: Automatic incremental view maintenance for rich query languages. In *SIGMOD*, 2023.

O. Polychroniou, A. Rokhlenko, and K. A. Ross. Arithmetic and comparisons for SIMD instructions. *PVLDB*, 8(9), 2015.

T. Willhalm, N. Popovici, Y. Boshmaf, J. Plattner, A. Zeier, and H. Schaffner. SIMD-scan: Ultra fast in-memory table scan. In *EDBT*, 2009 (also referenced in PVLDB 2009 proceedings).

A. Andoni and P. Indyk. Near-optimal hashing algorithms for approximate nearest neighbor in high dimensions. *Communications of the ACM*, 51(1), 2008.

P. Flajolet, E. Fusy, O. Gandouet, and F. Meunier. HyperLogLog: The analysis of a near-optimal cardinality estimation algorithm. In *Analysis of Algorithms (AOFA)*, 2007.

G. Cormode and S. Muthukrishnan. An improved data stream summary: The Count-Min sketch and its applications. *Journal of Algorithms*, 55(1), 2005.

A. Z. Broder. On the resemblance and containment of documents. In *SEQ/WCC*, 1997 (originated at STOC/WWW 1997).

B. Fan, D. G. Andersen, M. Kaminsky, and M. D. Mitzenmacher. Cuckoo filter: Practically better than Bloom. In *CoNEXT*, 2014.

J. Gomez-Luna, Y. Guo, S. Brocard, J. Legare, and O. Mutlu. Benchmarking memory-centric computing systems: Analysis of real processing-in-memory workloads. *IEEE Computer*, 56(8), 2023.

V. Seshadri, D. Lee, T. Mullins, H. Hassan, A. Boroumand, J. Kim, M. A. Zadeh, Y. Wang, A. Gibbons, S. Yazdanshenas, A. T. Malladi, B. Grot, T. H. M. Nguyen, S. A. Ebrahimzadeh, et al. Fulcrum: Accelerating PIM via fine-grained metadata caching. In *MICRO*, 2017.

S. Li, D. Niu, K. T. Malladi, H. Zheng, B. Brendan, and Y. Xie. DRISA: A DRAM-based reconfigurable in-situ accelerator. In *ISCA*, 2017.

F. R. Kschischang, B. J. Frey, and H.-A. Loeliger. Factor graphs and the sum-product algorithm. *IEEE Transactions on Information Theory*, 47(2), 2001.

P. D. Grünwald. A tutorial introduction to the minimum description length principle. In *Advances in Minimum Description Length*. MIT Press, 2004.

A. Wingo. Value representation in JavaScript implementations. Online engineering survey, 2011.

A. Schulze, N. E. de Barros Vasconcelos, G. D. Costa, R. F. P. de Oliveira, N. Lopes, J. L. P. da Silva, A. F. de S. Lima, D. T. R. de Macedo, P. C. M. da Silva, R. T. de Souza, T. T. T. Tomazela, A. K. de S. Ferreira, J. S. V. Pereira, and J. M. de M. P. dos Santos. ClickHouse — lightning fast analytics for everyone. *PVLDB*, 17(14), 2024.

D. Cantone and S. Faro. Twenty years of bit-parallelism in string matching. In *Festschrift for Udi Manber*. 2014 (survey, 2010s).